NAME: Pradeep Lahu Rathod

ROLL NO: 49

SUBJECT: AOA

EXPERIMENT NO.:7

To implement Kruskal's MST Algorithm using greedy method

Program code:

```
#include <stdio.h>
#include <stdlib.h>
#define MAX_EDGES 1000

typedef struct Edge {
   int src, dest, weight;
} Edge;

typedef struct Graph {
   int V, E;
   Edge edges[MAX_EDGES];
} Graph;

typedef struct Subset {
```

```
    int parent, rank;
} Subset;

Graph* createGraph(int V, int E) {
    Graph* graph = (Graph*) malloc(sizeof(Graph));
    graph->V = V;
    graph->E = E;
    return graph;
}

int find(Subset subsets[], int i) {
    if (subsets[i].parent != i) {
        subsets[i].parent = find(subsets, subsets[i].parent);
    }
    return subsets[i].parent;
}

void Union(Subset subsets[], int x, int y) {
    int xroot = find(subsets, x);
    int yroot = find(subsets, y);
      if (subsets[xroot].rank < subsets[yroot].rank) {
        subsets[xroot].parent = yroot;
```

```
    } else if (subsets[xroot].rank > subsets[yroot].rank) {
        subsets[yroot].parent = xroot;
    } else {
        subsets[yroot].parent = xroot;
        subsets[xroot].rank++;
    }
}

int compare(const void* a, const void* b) {
    Edge* a_edge = (Edge*) a;
    Edge* b_edge = (Edge*) b;
    return a_edge->weight - b_edge->weight;
}

void kruskalMST(Graph* graph) {
    Edge mst[graph->V];
    int e = 0, i = 0;
    qsort(graph->edges, graph->E, sizeof(Edge), compare);
    Subset* subsets = (Subset*) malloc(graph->V * sizeof(Subset));
    for (int v = 0; v < graph->V; ++v) {
        subsets[v].parent = v;
        subsets[v].rank = 0;
```

```
    }

    while (e < graph->V - 1 && i < graph->E) {
        Edge next_edge = graph->edges[i++];
        int x = find(subsets, next_edge.src);
        int y = find(subsets, next_edge.dest);
          if (x != y) {
            mst[e++] = next_edge;
            Union(subsets, x, y);
        }
    }
      printf("Minimum Spanning Tree:\n");
    for (i = 0; i < e; ++i) {
        printf("(%d, %d) -> %d\n", mst[i].src, mst[i].dest, mst[i].weight);
    }
}

int main() {
    int V, E;
    printf("Enter number of vertices and edges: ");
    scanf("%d %d", &V, &E);
    Graph* graph = createGraph(V, E);
```

```
    printf("Enter edges and their weights:\n");
   for (int i = 0; i < E; ++i) {
   scanf("%d %d %d", &graph->edges[i].src, &graph->edges[i].dest, &graph->edges[i].weight);
   }
   kruskalMST(graph);
   return 0;
}
```

Output -

```
Enter number of vertices and edges: 5 7
Enter edges and their weights:
0 1 2
0 3 6
1 3 8
3 4 9
1 4 5
1 2 3
2 4 7
Minimum Spanning Tree:
(0, 1) -> 2
(1, 2) -> 3
(1, 4) -> 5
(0, 3) -> 6


...Program finished with exit code 0
Press ENTER to exit console.
```